Section 6.2 — AutoGen

Multi-Agent Conversations with Direct Agent-to-Agent Communication

Author: Suresh D R | AI Product Developer & Technology Mentor

---

Table of Contents

#	Topic
6.2.1	What Is AutoGen
6.2.2	AutoGen vs LangGraph — The Key Difference
6.2.3	Core Components
6.2.4	How Agents Communicate
6.2.5	Conversation Patterns
6.2.6	Real-World Use Cases That Require AutoGen
6.2.7	When to Use AutoGen vs LangGraph
6.2.8	Section Summary

6.2.1 What Is AutoGen

AutoGen is Microsoft's open-source framework for building multi-agent systems where **agents talk to each other directly** — like a group chat between AI specialists.

Each agent has a name, a role, and a system prompt that defines its personality and expertise. When the conversation starts, agents exchange messages, critique each other's work, and refine outputs until a conclusion is reached — all automatically.

> **In one sentence:** AutoGen turns a task into a structured conversation between specialist agents who debate, critique, and improve each other's work until the job is done.

6.2.2 AutoGen vs LangGraph — The Key Difference

This is the most important concept before learning AutoGen.

> **LangGraph:** agents communicate through shared state. No agent ever calls another agent directly.
>
> **AutoGen:** agents communicate through direct messages. Agent A sends a message to Agent B. Agent B reads it and replies directly back.

LangGraph way — agents talk through state:

```
Research Agent  →  writes facts to STATE
                          ↓
               Plan Agent reads facts FROM STATE
               Plan Agent  →  writes plan to STATE
```

AutoGen way — agents talk directly to each other:

```
Research Agent  →  sends message directly to Plan Agent
Plan Agent      →  reads the message, replies directly
Research Agent  →  reads the reply and continues
```

In LangGraph — **state is the messenger.**

In AutoGen — **agents are the messengers.**

6.2.3 Core Components

AutoGen has four core building blocks. Understand these four and you understand AutoGen completely.

Component 1 — AssistantAgent

The AI brain. Powered by GPT. You give it a name and a system prompt that defines its role. The `name` appears in every message so you can see who said what. The `system_message` shapes how the agent thinks and responds throughout the conversation.

```
from autogen import AssistantAgent

analyst = AssistantAgent(
    name="Analyst",
    system_message=(
        "You are a senior logistics analyst. "
        "Analyse delivery data and identify patterns. "
        "Always provide data-driven conclusions with specific numbers."
    ),
    llm_config={
        "config_list": [{"model": "gpt-3.5-turbo", "api_key": "..."}]
    }
)
```

Component 2 — UserProxyAgent

Represents a human or an executor in the conversation. Has two key abilities — asking a real human for input, and running Python code that other agents write.

```
from autogen import UserProxyAgent

manager = UserProxyAgent(
    name="Manager",
    human_input_mode="NEVER",
    code_execution_config={"work_dir": "output", "use_docker": False},
    max_consecutive_auto_reply=10
)
```

human_input_mode controls when a real human gets involved:

Mode	Meaning
`NEVER`	Fully automated — no human involved
`ALWAYS`	Human reviews every single message
`TERMINATE`	Human steps in only when agent says TERMINATE

Component 3 — GroupChat

When you have more than two agents, GroupChat manages the multi-agent conversation. It holds all participating agents and controls conversation flow.

```
from autogen import GroupChat

group_chat = GroupChat(
    agents=[researcher, analyst, writer, critic],
    messages=[],      # starts empty, fills as conversation runs
    max_round=20      # stops after 20 rounds if no TERMINATE
)
```

Component 4 — GroupChatManager

The coordinator of a GroupChat. Reads the conversation history and decides which agent should speak next. Powered by an LLM so it makes intelligent decisions about who speaks based on context.

```
from autogen import GroupChatManager

chat_manager = GroupChatManager(
    groupchat=group_chat,
    llm_config={
        "config_list": [{"model": "gpt-3.5-turbo", "api_key": "..."}]
    }
)
```

6.2.4 How Agents Communicate

> **Analogy:** A team meeting. The manager gives the team a problem. The researcher speaks first and shares findings. The analyst responds with analysis. The writer drafts a solution. The critic challenges it. The writer improves it. Everyone reads what was said before — then responds directly to it. No whiteboard. No state dictionary. Direct conversation.

The Message Structure

Every message in AutoGen has three parts:

```
{
    "role":    "user" or "assistant",
    "content": "the actual message text",
    "name":    "which agent sent this"
}
```

When an agent speaks, its message is added to the shared conversation history. Every subsequent agent reads the **full history** before responding. This is how agents know what others said — by reading the conversation, not a state dictionary.

How the Conversation Grows — Step by Step

```
Round 1 — Manager:
  "Analyse Bandra delivery performance this week."

Round 2 — Researcher:
  "Bandra had 420 deliveries. 15% were delayed.
   Peak delay hours: 9–11 AM."

Round 3 — Analyst:
  "15% delay rate is above our 10% target.
   Root cause: not enough agents during 9–11 AM window."
```

```
Round 4 — Writer:
  "RECOMMENDATION: Add 8 agents in Bandra between 8:30–11:30 AM.
   Expected to reduce delays below 8%."

Round 5 — Critic:
  "Recommendation has no cost estimate.
   Please add cost analysis before finalising."

Round 6 — Writer:
  "UPDATED: Add 8 agents (Rs 12,000/day extra cost).
   Delay reduction: 15% to 8%. ROI achieved in 3 days."

Round 7 — Manager:
  "Approved. Implement from Monday. TERMINATE"
```

Each agent read every previous message before responding. The conversation built progressively toward a complete answer — that is AutoGen.

How AutoGen Decides Who Speaks Next

In a **two-agent conversation** — it alternates. Agent A speaks, Agent B replies, back and forth.

In a **GroupChat** — the GroupChatManager reads the conversation and decides who should speak next:

```
After Researcher shares raw data:
  Manager decides → "Analysis needed. Select Analyst."

After Analyst finds root cause:
  Manager decides → "Recommendation needed. Select Writer."

After Writer drafts a recommendation:
  Manager decides → "This should be challenged. Select Critic."

After Critic raises a concern:
  Manager decides → "Writer must address it. Select Writer again."
```

How a Conversation Ends

AutoGen conversations end in one of three ways:

1. Agent says TERMINATE — any agent can end the conversation by including TERMINATE in its message.

2. max_round is reached — conversation stops after the maximum rounds set in GroupChat.

3. Human ends it — if `human_input_mode="TERMINATE"`, the human can choose to end when the agent signals completion.

6.2.5 Conversation Patterns

AutoGen supports five distinct patterns. Each suits a different type of task.

Pattern 1 — Two-Agent Back-and-Forth

The simplest pattern. One AssistantAgent and one UserProxyAgent exchanging messages until the task is done.

Best for: simple tasks, code generation, single-domain problems.

```
Manager:   "Calculate fuel cost for 180 km, Rs 95/litre, 12 km per litre."
Assistant: "Fuel needed: 180 / 12 = 15 litres. Cost: 15 × 95 = Rs 1,425. TERMINATE"
```

Pattern 2 — Debate Pattern

Two agents argue opposing sides. One proposes, one critiques. They alternate until consensus is reached.

Best for: decisions that need to be challenged, quality control, risk assessment.

```
Proposer: "Add 10 agents to Andheri to reduce delays."
Critic:   "That costs Rs 15,000/day. Have you tried route optimisation first?"
```

```
Proposer: "Good point. Route optimisation first. If delay stays above 10% in 2 weeks, add 5 agents."
Critic:   "Better approach. I agree. TERMINATE"
```

Pattern 3 — Sequential Expert Chain

Multiple specialist agents each contribute expertise in a logical order — like passing work through a chain of experts.

Best for: tasks needing multiple types of expertise applied one after another.

```
Researcher  →  provides raw data and facts
     ↓
Analyst     →  analyses the data, identifies patterns
     ↓
Writer      →  writes the deliverable based on analysis
     ↓
Reviewer    →  checks quality and accuracy
     ↓
Manager     →  approves or requests changes
```

Pattern 4 — Round Table Discussion

Multiple agents all contribute to the same question. GroupChatManager decides who speaks based on what was just said. The conversation evolves organically.

Best for: brainstorming, strategy decisions, problems needing multiple angles.

```
Question: "Should we expand delivery to Pune next quarter?"

Data Analyst:   "Pune market: 2M potential customers, 0% current coverage."
Operations Mgr: "Expansion needs 50 agents, 3 warehouses. Cost: Rs 80L setup."
Risk Officer:   "Market not validated. Demand uncertain. High setup cost risk."
CX Lead:        "Customer surveys show 65% demand from Pune. Strong positive signal."
Data Analyst:   "At 10% conversion — 130K customers. Breakeven at 6 months."
Operations Mgr: "6-month breakeven is acceptable. I support."
```

```
Risk Officer:   "Demand data reduces my risk concern. Conditional support."
Manager:        "Consensus reached. Proceed with phased expansion. TERMINATE"
```

Pattern 5 — Code Writing and Execution Loop

One agent writes code, another executes it, the output feeds back into the conversation, and the writer improves the code based on results.

Best for: data analysis, automation scripting, debugging.

```
Manager:   "Analyse data.csv and find the peak delay hour."

Coder:     [writes Python code to find peak delay hour]

Executor:  [runs the code]
           "Output: Peak delay hour: 9"

Coder:     "Peak is at 9 AM. Getting more detail."
           [writes code to describe 8–12 AM window]

Executor:  [runs the code]
           "count: 245, mean: 23.4 min, max: 87 min"

Coder:     "Analysis complete. Peak delay 9 AM, average 23.4 min. TERMINATE"
```

6.2.6 Real-World Use Cases That Require AutoGen

These are scenarios where agents must genuinely communicate — debate, argue, improve, and reach consensus. LangGraph cannot handle these because its agents communicate through state, not through conversation.

Use Case 1 — Investment Decision Debate

Scenario: A company is deciding whether to invest Rs 2 crore in expanding Pune delivery operations. The decision needs financial, operational, risk, and market perspectives — all challenging each other before a final call.

Why AutoGen: The CFO must hear the Risk Officer's concern and address it directly. Operations must challenge the Market Agent's demand assumption. This evolves based on what is said — it cannot be scripted into a fixed workflow.

Agents and Roles:

- CFO Agent — evaluates financial return and breakeven
- Risk Officer Agent — identifies and quantifies risks
- Operations Agent — assesses feasibility and timelines
- Market Agent — analyses demand and competition

Conversation:

```
Market Agent:    "Pune market has 65% demand signal. 2M addressable customers."
CFO Agent:       "At 10% conversion, Rs 2 crore breaks even in 8 months. Acceptable."
Risk Agent:      "Demand signal is survey-based, not actual purchase data. High uncertainty."
Operations:      "Setup needs 3 months. We miss the festive season if we start now."
CFO Agent:       "Missing festive season changes breakeven to 14 months. That changes my view."
Market Agent:    "But we lose first-mover advantage. Competitor expanding in Q3."
Risk Agent:      "First-mover in unvalidated market is not an advantage — it is a risk."
Operations:      "Compromise: pilot with 1 warehouse, 20 agents. Cost Rs 40L."
CFO Agent:       "Pilot at Rs 40L with option to expand. Lower risk, validates demand."
Risk Agent:      "Pilot approach is acceptable. I support."
Market Agent:    "Pilot is better than nothing. I support."
Manager:         "Pilot approved. Full expansion contingent on results. TERMINATE"
```

> Each agent responded to what the previous agent said. That is only possible in AutoGen.

Use Case 2 — Operations Report Review

Scenario: A logistics company publishes a monthly operations report. It needs to be written, fact-checked, edited, reviewed for legal compliance, and approved — with each reviewer able to send it back for revision.

Why AutoGen: The Legal reviewer may catch something requiring the Writer to rewrite. The Editor may simplify language that the Fact Checker then needs to re-verify. The conversation loops back based on what each reviewer finds.

Agents and Roles:

- Writer Agent — drafts the report
- Fact Checker Agent — verifies all claims against data
- Editor Agent — improves clarity and readability
- Legal Agent — checks for compliance issues
- Approver Agent — gives final sign-off

Conversation:

```
Writer:       [writes first draft of monthly report]
Fact Checker: "Claim: 95% on-time delivery. Actual data shows 91.3%. Please correct."
Writer:       [corrects the statistic to 91.3%]
Legal:        "Phrase 'guaranteed delivery' is a legal liability. Replace with 'target delivery'."
Writer:       [makes the legal correction]
Editor:       "Paragraph 3 is too technical for business readers. Please simplify."
Writer:       [simplifies paragraph 3]
Fact Checker: "New paragraph 3 says 'most deliveries on time'. Needs the specific percentage."
Writer:       [adds 91.3% back in]
Editor:       "Now clear and accurate."
Legal:        "No further issues."
Approver:     "Report approved for publication. TERMINATE"
```

Use Case 3 — Software Architecture Review

Scenario: An engineering team is designing a new delivery tracking system. Architects propose designs, engineers critique them, security flags vulnerabilities, and a lead makes the final call.

Why AutoGen: A security concern raised by one agent changes what the architect proposes. The performance engineer may disagree with the security engineer's trade-off. These are genuine technical discussions — not fixed pipeline steps.

Agents and Roles:

- Architect Agent — proposes the system design
- Backend Agent — evaluates backend feasibility
- Security Agent — identifies vulnerabilities
- Performance Agent — evaluates speed and scalability
- Lead Agent — makes the final architectural decision

Conversation:

```
Architect:   "Proposed: REST API + PostgreSQL + Redis cache + JWT auth."
Backend:     "Solid foundation. But JWT refresh for 500K users could become a bottleneck."
Security:    "JWT without short expiry is a risk. Need 15-minute expiry with refresh tokens."
Performance: "15-min expiry means 33K refresh calls/minute at peak for 500K users."
Architect:   "Revised: JWT 15-min + refresh token stored in Redis. Redis handles 100K ops/sec."
Backend:     "Redis for refresh adds complexity but is the right approach."
Security:    "Acceptable if Redis is encrypted at rest."
Performance: "Redis handles the load comfortably. No concerns."
Lead:        "Architecture approved: REST + PostgreSQL + Redis + JWT 15-min. TERMINATE"
```

Use Case 4 — HR Policy Decision

Scenario: A logistics company needs to decide a new attendance policy for 500 delivery agents — balancing agent fairness, operational reliability, and legal compliance.

Why AutoGen: HR decisions require balancing genuinely competing interests. Each party responds to the others' specific points — the policy evolves through real discussion.

Agents and Roles:

- HR Manager Agent — focused on fairness and agent retention
- Operations Agent — focused on delivery reliability
- Legal Agent — focused on labour law compliance
- Finance Agent — focused on cost implications

Conversation:

```
HR Manager:  "Proposal: agents can take 2 unplanned leaves per month without penalty."
Operations:  "2 unplanned absences per agent means 60+ daily shortfalls at peak. Unworkable."
HR Manager:  "Restricting leaves causes attrition. We lose agents anyway if we are too strict."
Legal:       "Under Shop and Establishment Act, at least 1 casual leave per month is mandatory."
Finance:     "Every absent agent costs Rs 800 in replacement. 2/month × 500 agents = Rs 8L/month."
Operations:  "Compromise: 1 unplanned leave + 1 advance-notice leave per month."
HR Manager:  "1 unplanned is the minimum I can accept. Agreed."
Legal:       "This meets legal minimum requirements."
Finance:     "Rs 4L/month vs Rs 8L/month. Acceptable."
Manager:     "Policy approved: 1 unplanned + 1 advance-notice leave per month. TERMINATE"
```

Use Case 5 — Customer Escalation Resolution

Scenario: A high-value customer lost Rs 2L of perishable goods due to a 4-hour delivery delay. Multiple departments must agree on what to offer — balancing customer satisfaction, legal risk, and business value.

Why AutoGen: Legal restricts what Support wants to offer. Operations provides facts that change what the Account Manager proposes. The resolution emerges through agents reading each other's constraints and finding a solution that satisfies all parties.

Agents and Roles:

- Support Agent — advocates for customer satisfaction
- Operations Agent — explains what happened operationally
- Legal Agent — sets boundaries on what can be offered
- Account Manager Agent — manages the customer relationship

Conversation:

```
Support:     "Customer lost Rs 2L of goods due to our 4-hour delay. Demanding full refund."
Operations:  "Delay caused by vehicle breakdown. SLA allows 2-hour delay before compensation triggers."
Legal:       "Full refund exposes us to consequential damages liability. SLA cap: Rs 50K maximum."
Support:     "Rs 50K is 25% of their loss. Customer will not accept. They will churn."
Acct Mgr:   "This customer does Rs 40L/year with us. Losing them costs far more than Rs 2L."
Legal:       "Rs 1L as ex-gratia payment — not admission of liability — is permissible."
Support:     "Rs 1L ex-gratia + Rs 25K service credit for next month = Rs 1.25L total offer."
```

```
Acct Mgr:   "With a personal VP apology call, they will stay."
Legal:       "Approved if worded as ex-gratia with no admission of liability."
Manager:     "Offer approved: Rs 1L ex-gratia + Rs 25K credit + VP call. TERMINATE"
```

6.2.7 When to Use AutoGen vs LangGraph

Situation	AutoGen	LangGraph
Agents need to debate and argue	✅	❌
Agents respond to each other's specific points	✅	❌
Number of conversation turns is unpredictable	✅	❌
Multiple perspectives challenging each other	✅	❌
Code writing and execution loops	✅	❌
Structured workflow with fixed stages	❌	✅
Conditional branching based on state	❌	✅
Crash recovery and checkpointing	❌	✅
Human approval at a specific node	❌	✅
State must persist across sessions	❌	✅

The Simple Decision Rule

> **"Can agents figure out what to say by reading the conversation?"**
> → Use **AutoGen**

> **"Can agents figure out what to do by reading the state?"**
> → Use **LangGraph**

6.2.8 Section Summary

AutoGen is built for tasks where the path to the answer is **discovered through conversation** — not scripted in advance.

The Four Core Components

Component	What It Is	When You Use It
AssistantAgent	AI agent with a role and system prompt	Every agent that uses an LLM
UserProxyAgent	Human representative or code executor	Entry point for tasks, code runs
GroupChat	Container for multi-agent conversation	When 3 or more agents need to talk
GroupChatManager	LLM-powered conversation moderator	Required for every GroupChat

---

The Five Conversation Patterns

Pattern	Best For
Two-agent back-and-forth	Simple tasks, code writing
Debate	Decisions that need to be challenged
Sequential expert chain	Multi-expertise tasks in order
Round table discussion	Brainstorming, strategy, complex decisions
Code write and execute	Data analysis, automation, debugging

The Key Rule

In AutoGen — agents read the **conversation**. They respond to what was **said**.

In LangGraph — agents read the **state**. They respond to what was **written**.

This single difference determines which framework to use. If your task benefits from agents genuinely responding to each other — use AutoGen. If your task benefits from a controlled, resumable, branchable workflow — use LangGraph.

Suresh D R — AI Product Developer & Technology Mentor
Agentic AI — Section 6.2 AutoGen | Generative AI Training Course